

NOHOLI LIBRARY MANAGEMENT SYSTEM

Backend Team Handoff — What Has Been Done, What Is Being Provided, What Backend Must Build, and What Must Be Fixed

Purpose	Give the backend team a clear picture of the completed frontend/product work, handoff materials, production backend responsibilities, and known risks/gaps.
Frontend platform	Lovable / Supabase-style frontend implementation
Authoritative catalogue	All Book List.xlsx — 3,016 book records
Production starting point	3,016 books / 8,086 total copies
Important	Current Lovable Cloud records are QA/development records, not the production dataset.

Read this together with the Backend Requirements PDF, GitHub repository, and All Book List.xlsx.

1. What I Did as the UX/UI + Frontend Owner

I designed and implemented the Noholi Library Management System frontend in Lovable, with the goal of establishing the actual information architecture, workflows, terminology, interaction behavior, and frontend data expectations before production backend implementation.

Product structure and UX

- Established Dashboard, Inventory, Lending, Members, Donations / Book Donations, Fines, Reports, and Settings.
- Defined the product around speed, accuracy, low cognitive effort, readable data, and practical administration.
- Separated Inventory, Lending, Members, Donations, and Fines into related operational areas.
- Defined Genre vs Category; Category represents audience classification such as Adults, Children, and General.
- Designed for a single-admin product model with protected administrative data.

Inventory

- Cleaned the catalogue to the production starting set of 3,016 records.
- Defined bilingual title/author fields, genre, publication, year, edition, language, category, ISBN, copies, condition, pages, price, and cover reference.
- Implemented search/filtering, details, edit workflow, availability/status presentation, and system-derived copy counts.
- Verified Inventory Edit: correct loading, editable metadata, read-only calculated fields, save/feedback, table update, and persistence after refresh.
- Defined Available Copies = Total Copies – Issued Copies – Reserved Copies.

Lending

- Designed the 3-step borrowing flow: Select Book → Guarantor Details → Confirm Request.
- Implemented manual guarantor capture, Issue Book, Active Loans, return, cancellation/void, due-date editing, guarantor editing, and notes.
- Defined loan statuses: Active, Returned, Overdue, Cancelled, Lost.
- Tested issue, return, double-return, no-available-copy, cancellation/void, persistence after refresh, and inventory count changes.

Members, Fines, and Donations

- Implemented Member CRUD and Member View with contact, membership, activity, and history areas.
- Implemented manual Merit Grade A/B/C/D/E/Not Assigned with optional Merit Note; no automatic merit scoring.
- Implemented donation workflow: Pending → Approved → Added to Inventory, or Pending → Rejected.
- Verified donation rejection with required reason and verified approved donation creation of a real Book ID.
- Implemented the fines area around real overdue/fine/payment data rather than demo rows.
- Defined member history requirements for loans, overdue activity, fines/payments, reservations where applicable, and optional merit history.

2. What Has Been Tested

Important workflows were exercised against the working frontend and connected data. These tests demonstrate frontend behavior; they do not replace backend transaction, constraint, or security testing.

Area	Result	Backend implication
Inventory Edit	PASS	Production DB constraints/RLS still required.
Loan Issue	PASS	Inventory + loan must be atomic.
Loan Return	PASS	Backend transaction still required.
Double Return	PASS	Must remain protected under concurrency.
No Available Copy	PASS	DB/server logic must enforce it.
Loan Cancel/Void	PASS	Historical record should be preserved.
Member View / Merit	PASS	History must derive from production data.
Donation Reject	PASS	Reason required; record retained.
Donation Approve + Add	PASS	DB duplicate prevention + transaction required.
Fines	Implemented	Exact production fine-rate policy still needs confirmation.

Donation QA created DON-002 and inventory record BK-3017, temporarily making the Cloud QA state 3,017 books / 8,087 copies. These are QA records and must NOT be imported into production.

3. What I Am Providing to the Backend Team

Item	Purpose	Instruction
GitHub repository	Frontend source, UI implementation, Supabase client/types, migrations and project structure.	Use as frontend contract/reference; it is not a complete production DB schema.
All Book List.xlsx	Authoritative initial inventory catalogue.	Import 3,016 records; this is the production starting source.
Backend Requirements PDF	Business rules, entities, security, transactions, invariants, testing and acceptance criteria.	Use as the functional backend contract.
This Handoff Summary PDF	Explains work completed, handoff materials, backend ownership, and risks.	Use as orientation for implementation.

The current Lovable Cloud database is a development/QA state. Build a clean production database rather than assuming the Cloud state can be reproduced from GitHub.

Repository / database situation

- The repository contains frontend source, TypeScript Supabase types, the Excel catalogue, and some migration files.
- It does not contain a complete reproducible production schema with all CREATE TABLE statements, RLS policies, grants, constraints, and complete live data.
- The live Cloud database contains QA/test records that are not equivalent to the original catalogue.
- Therefore the backend team needs to create a clean production schema and migrations.

4. What the Backend Team Needs to Build

The backend team should not redesign the UX. The frontend establishes the intended workflows. Backend must make those workflows persistent, secure, transactional, reproducible, and safe under real production use.

A. Production database

- Create production schema for books/inventory, members, loans, fines, fine payments, donations, and reservation/audit entities if in final scope.
- Define exact PostgreSQL types, primary/foreign keys, unique/check constraints, indexes, defaults, timestamps, and deletion behavior.
- Create versioned migrations so a fresh environment can be built without Lovable Cloud.
- Import and verify the 3,016-book Excel catalogue.

B. Transactions / business logic

- Loan issue: atomically validate availability, create loan, and update inventory.
- Loan return: atomically close loan and release inventory.
- Loan cancellation/void: release inventory where appropriate while retaining history.
- Donation Add to Inventory: create exactly one inventory book and update donation in one transaction.
- Payment recording: prevent duplicate payments and keep fine status consistent.
- Enforce business rules at database/server level, not only by hiding frontend buttons.

C. Authentication / security

- Protect all administrative operations behind authenticated authorization.
- Use RLS and/or equivalent server-side authorization.
- Public users may see only intentionally public book fields.
- Member, guarantor, loan, fine, payment, and donation data must remain private.
- Never expose service-role credentials or database secrets in frontend/GitHub.
- Re-establish production password/security policy rather than preserving a weak development credential.

5. Core Production Data Rules

Area	Required rule
Inventory	Available = Total – Issued – Reserved. Available cannot be negative; Issued cannot exceed Total.
Book identity	Unique system Book ID using the product's BK-#### convention.
Members	Unique Member ID; status Active/Suspended/Expired; merit A/B/C/D/E/Not Assigned.
Loans	Every loan references a real Member ID and Book ID; issue/return/cancel are transactional.
Fines	Fine records relate to actual loans/members; payment and waiver actions are authorized and persistent.
Donations	Pending → Approved → Added to Inventory, or Pending → Rejected; rejection requires a reason.
Donation inventory	Added-to-Inventory donation must have exactly one valid Inventory Book ID; duplicate addition blocked at DB level.
History	Important operational history is preserved; use cancellation/void/status semantics rather than destructive deletion.

6. Problems / Risks That Need to Be Fixed Before Production

These are engineering risks/gaps. They do not mean the frontend is unusable; they identify what must be strengthened or clarified before launch.

Risk / gap	Why it matters	Backend action
No complete reproducible schema in repo	Fresh environments cannot safely recreate DB from Git alone.	Create full versioned schema/migrations.
RLS / authorization needs full audit	Sensitive member, guarantor, loan, fine and donation data must not be public.	Review every table and operation.
Frontend success is not transaction safety	Concurrent requests can corrupt inventory if logic is frontend-only.	Use DB transactions, atomic updates, constraints/locking.
Fine policy not fully specified	Backend needs exact calculation rules.	Confirm daily BDT rate, grace period, caps/exceptions.
Reservations not fully specified	Reserved Copies exist, but full reservation workflow is not defined.	Confirm scope and define schema/rules if included.
Image/storage strategy	Excel has cover URLs; storage architecture is separate.	Decide URL-only vs managed storage; secure uploads if used.
Cloud contains QA records	QA data must not accidentally enter production.	Start from 3,016 Excel records / 8,086 copies.
Development auth history	Security controls were relaxed during development.	Create fresh production admin and restore strong policy.
API/RPC contract needs technical detail	Frontend types do not replace backend implementation.	Define inputs, outputs, errors, auth rules and indexes.

7. Questions to Resolve Before the Schema Is Locked

- Fine calculation: exact BDT daily overdue rate, grace period, maximum, and treatment of lost books.
- Reservations: whether they are part of the first production release and their lifecycle/counting rules.
- Donation relationship: whether the created Book should directly store the originating Donation ID.
- Book identity: confirm that each inventory row represents a title-level record with a copy count rather than one row per physical copy.
- Images: whether existing external cover URLs remain authoritative or images move into controlled storage.
- Reports: which calculations need dedicated backend views/queries.
- Audit: which administrative actions require an audit log.

8. Production Starting Point

Dataset	Production instruction
Books	Import exactly 3,016 records from All Book List.xlsx.
Total copies	Initial source total: 8,086.
ISBN	Currently blank; keep optional.
Cover	Source contains Cover Image URL values; Cloud storage state is not the source catalogue.
QA records	Do NOT import BK-3017, MEM-1001, DON-001, DON-002 or other temporary Cloud QA records.
Admin	Create a fresh production admin account; do not migrate development credentials.

9. Backend Definition of Done

The backend handoff should be considered complete only when these are demonstrably true:

- Fresh database can be created entirely from versioned migrations.
- 3,016 source books import correctly and totals reconcile to 8,086 copies.
- RLS/authorization passes authenticated and unauthorized-access tests.
- Issue, return, cancel/void, and no-copy scenarios are transactional.
- Concurrent loan operations cannot create negative or over-issued inventory.
- Donations cannot be added twice, including through concurrent requests.
- Fine payments cannot be duplicated and waivers require authorized reasons.
- Member history comes from actual relational records.
- Sensitive data is not exposed through anonymous/public queries.
- Backup and restore procedure is documented and tested.
- Secrets are separated from source control.
- Frontend connects to production without Lovable Cloud-only assumptions.

10. Final Handoff Summary

What I have done

- Designed the product structure and UX for Noholi Library Management System.
- Built the working frontend/admin workflows in Lovable.
- Established inventory, lending, member, fine, donation, authentication, and history requirements.
- Tested important workflows against the working frontend and connected data.
- Established the authoritative initial inventory source of 3,016 books.
- Connected the project to GitHub and reviewed the repository/database situation.
- Prepared the backend requirements as a functional contract.

What I am providing

- GitHub repository containing the frontend/application source.
- All Book List.xlsx containing the authoritative 3,016-book starting catalogue.
- Backend Requirements PDF containing the detailed functional/backend contract.
- This Handoff Summary PDF explaining completed work, handoff materials, backend responsibilities, and production risks.

What the backend team owns

- Production PostgreSQL/Supabase schema and migrations.
- RLS, authorization, grants, and secure access.
- Transactional loan/inventory logic.
- Fine/payment/waiver persistence and duplicate protection.
- Donation/inventory transaction and duplicate prevention.
- Server-side validation, constraints, indexes and API/RPC contracts.
- Backups, restore, deployment configuration, audit needs, and production integrity.

Bottom line: The frontend/product work has established and demonstrated the intended workflows. The remaining critical work is production backend engineering: reproducible database schema, secure authorization, transactional business logic, exact technical contracts, and resolution of the few business-rule questions identified above.

Prepared for the Noholi Library Management System backend handoff.